

BRUTE FORCE ATTACK ON SSH

DETECTION SUMMARY AND PROTECTION IMPLEMENTATION GUIDE

by MIR MUHAMMED AHSAN ALI

```
root@adc1:~# journalctl _SYSTEM_UNIT=ssh.service | egrep 'Failed|failure'
Dec 05 21:39:17 ac1 sshd[41458]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
tty=ssh ruser= rhost=192.168.1.3 user=root
Dec 05 21:39:17 ac1 sshd[41458]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
tty=ssh ruser= rhost=192.168.1.3 user=root
Dec 05 21:39:20 ad1 sshd[41458]: Failed password for root from 192.168.1.3 port 37362 ssh2
Dec 05 21:39:23 ad1 sshd[41458]: Failed password for root from 192.168.1.3 port 37362 ssh2
Dec 05 21:39:28 ad1 sshd[41458]: Failed password for root from 192.168.1.3 port 37362 ssh2
Dec 05 21:39:28 ad1 sshd[41458]: Failed password for root from 192.168.1.3 port 37362 ssh2
```

Date: March 28, 2026

Executive Summary

On March 27, 2026, the **sm40.com** web server was subjected to a brute-force attack targeting the SSH service. The attack successfully overwhelmed the web server and ultimately rendered the origin server unresponsive. The host became unreachable, resulting in HTTP 522 errors for visitors.

This report summarises the incident based on information provided by an external consultant and a screenshot. It also provides clear, actionable recommendations to prevent recurrence, specifically the deployment of Cloudflare Tunnel, Fail2Ban, SSH rate limiting, and strengthened firewall rule.

Contents

Incident Overview

- What was the issue?
- Root Cause Analysis
- Post incident mitigation

Recommended Mitigation Strategy

- Why is Cloudflare CDN a paper shield to attackers?
- Recommendation Strategy
 - Fail2Ban
 - Advanced Firewall Rules
 - Combined Security
 - Cloudflare Tunnel

Conclusion

Incident Overview

What was the issue?

An external attacker performed a brute-force login attack against the SSH service. When I visited the website during the incident, the following symptoms were visible:

- HTTP error code **522** (connection timed out)
- Cloudflare edge was fully operational and returning responses.
- The error explicitly mentioned **Host Error**.

The external consultant Imon Vai was informed by the server administrator of the incident and he reviewed the SSH authentication logs and confirmed the attack vector.

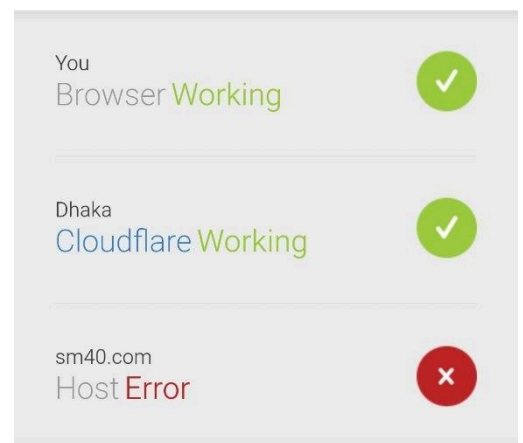


Connection timed out

Error code 522

Visit cloudflare.com for more information.

2026-03-27 07:12:54 UTC



What happened?

The initial connection between Cloudflare's



Root Cause Analysis

Exposed SSH port- Port 22 was publicly reachable on the internet with no additional protection layers.

No rate limiting- No mechanism to slow down or block rapid login attempts.

No automatic ban system- Failed attempts were logged but not acted upon in real time. The attack overwhelmed server resources resulting in the server being unable to respond to the real requests.

Post incident mitigation

After analyzing the incident. The initiative of hosting the website using Cloudflare CDN was taken to hide the origin host IP address from the attacker.

Recommended Mitigation Strategy

Why is Cloudflare CDN a paper shield to attackers?

Although the origin server is hiding behind Cloudflare CDN, a hacker can easily find the origin server because the server left its footprint on the internet. There are some popular DNS histories where the origin server IP is visible.

e.g. viewdns.info

HTML JSON  

IP history results for 'sm40.com'.

IP ADDRESS	LOCATION	IP ADDRESS OWNER	LAST SEEN ON THIS IP
172.67.179.227	Unknown	Cloudflare, Inc	2026-03-27
104.21.35.213	Unknown	Cloudflare, Inc	2026-03-27
10.10.10.10	Bangladesh	[REDACTED]	2026-03-26
10.10.10.10	Bangladesh	[REDACTED]	2024-07-31
10.10.10.10	Stockholm - Sweden	AS40676	2023-03-07
19.19.19.19	United States	FASTLY	2023-02-02
10.10.10.10	Bangladesh	Stargate Communications Ltd.	2022-04-30
10.10.10.10	Singapore - Singapore	PhoenixNAP	2021-11-08

Post Incident Mitigation

The origin IP

Visible Footprint in History

```

HostHunter: v1.6
Author : Andreas Georgiou (@superhedgy)

[+] Target: 1[REDACTED]
[+] Hostnames:
[REDACTED] sm40.com
[REDACTED] com
[REDACTED] .com
[REDACTED]
[REDACTED] com
[REDACTED] .com
[REDACTED] com
[REDACTED] 40.com
[REDACTED] m
[REDACTED] m
[REDACTED] om

Reconnaissance Completed!
[REDACTED] hostnames were discovered in 1.82 sec

```

As you can see, the list of previous IP addresses are collected from the internet. Even if the IP addresses have hosted sm40.com long ago. They got documented.

From an attacker's perspective, the top one IP must be the origin IP. Since the owner of the first two is cloudflare then the third one must be the origin IP.

And the assumption was proven true by using a linux tool **HostHunter**.

The tool ensured that this origin server is actually **hosting sm40.com right now**.

Recommendation Strategy

The following layered inexpensive security controls will dramatically reduce the risk of future SSH brute-force attacks:

Recommendation	Purpose
Fail2Ban	Automatically bans IPs after a configurable number of failed SSH login
Firewall Rules	Deny SSH login attempts from unknown IPs
Combined Security	Restrict SSH to trusted IPs only + Rate limiting + integrate with Fail2Ban
Cloudflare Tunnel	Hides the real server IP and SSH port behind Cloudflare's global network (zero-trust access)

Fail2Ban

Fail2ban is **The Automated Jailer** which is a robust Python-based daemon that protects Linux servers from brute-force attacks by scanning log files (/var/log/auth.log) for excessive failed login attempts. It constantly reads the log files and looks for specific patterns like multiple “Failed password for root” and automatically bans the malicious IPs by updating firewall rules (iptables, firewalld, or ufw) for a set time.

It is better than a firewall because a firewall is static. Fail2Ban is **reactive**. It catches attackers who are trying to be "sneaky" by staying just **under the rate limit**.

Modern day hackers are genius and can bypass rate limiting by brute-forcing with low rate. Fail2ban looks for multiple failed attempts(you have to customize the number of failed attempts).

Read more: <https://github.com/fail2ban/fail2ban>

You can find the installation guide in the provided link.

Configure the jail.local file:

port - Which port to look after. e.g. ssh.

logfile - the location of the log file. e.g. /var/log/auth.log

bantime - How long the IP remains blocked

findtime - The time window Fail2Ban looks back into the logs.

maxretry - The number of failed attempts allowed before the ban is triggered.

Advanced Firewall Rules

Firewall rules can be integrated to allow only whitelisted IPs to access the server. This can be done using several tools in debian servers.

The firewall tracks how many connection attempts come from a single IP. You can implement basic rate limiting so that, if an IP tries to connect more than, lets say, 6 times in 30 seconds, the firewall blocks them temporarily.

Read more for **detailed instruction**: [How to Set Up UFW Rate Limiting for SSH](#)

The installation guide and usage of a popular firewall tool **ufw** is given here: [How to limit SSH access by IP addresses using firewall](#)

CAUTION : When customizing the rule, a "Deny All" must come **after** "Allow" rules, (the allow rule is the whitelist of IPs) , or he will be locked out immediately.

Combined Security

A **Combined Firewall and Fail2Ban** protects the server from brute-force attack on SSH service. Standard rate limiting using a firewall (like "5 attempts per minute") often just slows an attacker down, but they can still keep trying forever. When you use these two together, you move from just "slowing them down" to **completely removing their access**.

Feature	Simple Rate Limiting	Firewall + Fail2Ban
Reaction	Slows down the attacker.	Completely blocks the attacker's IP.
Visibility	The attacker can still see the port is open.	The attacker sees the server as "Down" or "Dead."
Bypass Potential	High (just wait for the timer).	Low (requires a new IP address for every 3 tries).
Resource Usage	High (server still processes the request).	Very Low (Firewall drops traffic instantly).

Cloudflare Tunnel

A **Cloudflare Tunnel** is a private, outbound-only connection between your server and the Cloudflare network. It allows you to expose services (like SSH) to the internet **without opening any ports** on your firewall or having a public IP address.

You can put a login screen (Google, GitHub, OTP) in front of your SSH so only *you* can even see the login prompt. All traffic between your server and the Cloudflare edge is encrypted.

Learn more about it: [Cloudflare Tunnel](#)

The instructions : [SSH with Access for Infrastructure](#)

- You need to install a software "cloudflared" in the server for it.
- Close Port 22 on your firewall. so it cannot be seen and cannot be brute-forced.

- Configure a "Public Hostname" so that you can connect to ssh via **████████.sm40.com**.
- map the subdomain to ssh://localhost:22 .
- Create a Policy mentioning which email addresses are allowed.
- Add google identity provider. (Zero Trust Access)
- Activate 2 factor authentication.

Make sure to keep the DNS record for www.sm40.com pointing to the server's IP address. So the HTTP/HTTPS users can visit the website as they are doing right now. So even though the origin IP is public. But the only way of getting in via SSH is secured by cloudflare zero trust. (tunnel + 2FA)

More detailed guide here: [Cloudflare Tunnel SSH Setup Guide](#)

Conclusion

The recent SSH brute-force attack was done because the SSH service was directly exposed to the internet. Even though Cloudflare CDN is used, it can't hide the origin server anyway. By implementing the provided features, the server will be significantly hardened against future attacks while keeping management simple and secure.

First deploy Fail2ban and UFW Firewall to provide an immediate, automated defense against ongoing brute-force attacks at the server level. (Takes 30 min)

Once this baseline security is active, we will transition to a Cloudflare Tunnel to completely hide the server from the public internet and move toward a Zero Trust architecture. (Might take more than an hour)

Please feel free to reach me for any relevant issues or any corrections. Glad to be part of the community.

Also, please patch the previously reported issues too.

- Mir Muhammed Ahsan Ali
ali.codes1279@gmail.com
t.me/M999Ali